

Tipo de datos mapa: diccionario

Los diccionarios son tipos de datos que nos permiten guardar valores, a los que se puede acceder por medio de una clave (una cadena de caracteres o un número). Son tipos de datos mutables y los campos no tienen asignado orden.

Definición de diccionarios.

```
>>> diccionario = {'one': 1, 'two': 2, 'three': 3}
```

Si tenemos un diccionario vacío, al ser un objeto mutable, también podemos construir el diccionario de la siguiente manera.

```
>>> dict1 = {}
>>> dict1["one"]=1
>>> dict1["two"]=2
>>> dict1["three"]=3
```

Operaciones básicas con diccionarios

```
>>> a = {'one': 1, 'two': 2, 'three': 3}
```

- `len()`: Devuelve número de elementos del diccionario.

```
>>> len(a)
3
```

- Indexación: Podemos obtener el valor de un campo o cambiarlo (si no existe el campo nos da una excepción `KeyError`):

```
>>> a["one"]
1
>>> a["one"]+=1
>>> a
{'three': 3, 'one': 2, 'two': 2}
```

- `del()`: Podemos eliminar un elemento, si no existe el campo nos da una excepción `KeyError`:

```
>>> del(a["one"])
>>> a
{'three': 3, 'two': 2}
```

- Operadores de pertenencia: `key in d` y `key not in d`.

```
>>> "two" in a
True
```

Los diccionarios son tipos mutables

Los diccionarios, al igual que las listas, son tipos de datos mutable. Por lo tanto podemos encontrar situaciones similares a las que explicamos en su momento con las listas.

```
>>> a = {'one': 1, 'two': 2, 'three': 3}
>>> a["one"]=2
>>> del(a["three"])
>>> a
{'one': 2, 'two': 2}
```

```
>>> a = {'one': 1, 'two': 2, 'three': 3}
>>> b = a
>>> del(a["one"])
>>> b
{'three': 3, 'two': 2}
```

En este caso para copiar diccionarios vamos a usar el método `copy()`:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = a.copy()
>>> a["one"]=1000
>>> b
{'three': 3, 'one': 1, 'two': 2}
```

Métodos principales de diccionarios

Métodos de eliminación

`clear()`: Elimina los elementos de un diccionario.

```
>>> dict1 = {'one': 1, 'two': 2, 'three': 3}
>>> dict1.clear()
>>> dict1
{}
```

Métodos de agregado y creación

`copy()`: Como hemos explicado anteriormente nos permite copiar diccionarios.

```
>>> dict1 = {'one': 1, 'two': 2, 'three': 3}
>>> dict2 = dict1.copy()
```

`update()`: Nos permite añadir elementos a un diccionario a partir de los elementos de otro diccionario.

```
>>> dict1 = {'one': 1, 'two': 2, 'three': 3}
>>> dict2 = {'four':4,'five':5}
```

```
>>> dict1.update(dict2)
>>> dict1
{'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5}
```

Métodos de retorno

`get()`: Nos devuelve el valor de un elemento de un diccionario indicando la clave. Además podemos indicar el valor devuelto si no existe el elemento.

```
>>> dict1 = {'one': 1, 'two': 2, 'three': 3}
>>> dict1.get("one")
1
>>> dict1.get("four")
>>> dict1.get("four", "no existe")
'no existe'
```

`pop()`: Nos devuelve el valor de un elemento de un diccionario a partir de la clave y elimina dicho elemento. También podemos indicar el valor devuelto si no existe el elemento buscado.

```
>>> dict1.pop("one")
1
>>> dict1
{'two': 2, 'three': 3}
>>> dict1.pop("four")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'four'
>>> dict1.pop("four", "no existe")
'no existe'
```

Recorrido de diccionarios

El método `keys()` nos devuelve las claves de un diccionario, por lo tanto podemos recorrer las claves de la siguiente manera:

```
>>> for clave in dict1.keys():
...     print(clave)
one
two
three
```

El método `values()` nos devuelve los valores de un diccionario, por lo tanto podemos recorrer los valores de la siguiente manera:

```
>>> for valor in dict1.values():
...     print(valor)
1
2
3
```

Finalmente el método `items()` nos devuelve la clave y el valor correspondiente de los elemento de un diccionario, por lo tanto podemos recorrer ambos:

```
>>> for clave,valor in dict1.items():
...     print(clave,"->",valor)
one -> 1
two -> 2
three -> 3
```

Excepciones

Errores sintácticos y errores de ejecución

Los errores sintácticos se producen cuando tenemos algo mal escrito en nuestro código fuente. Veamos un ejemplo de error sintáctico:

```
>>> while True print('Hello world')
      File "<stdin>", line 1
        while True print('Hello world')
                        ^
SyntaxError: invalid syntax
```

Una excepción o un error de ejecución se produce durante la ejecución del programa. Las excepciones se puede manejar para que no termine el programa. Veamos algunos ejemplos de excepciones:

```
>>> 4/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero

>>> a+4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined

>>> "2"+2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't convert 'int' object to str implicitly
```

Hemos obtenido varias excepciones: `'ZeroDivisionError'`, `'NameError'` y `'TypeError'`. Puedes ver la [lista de excepciones](#) y su significado.

Manejando excepciones

Veamos un ejemplo simple como podemos tratar una excepción:

```
>>> while True:
...     try:
...         x = int(input("Introduce un número:"))
```

```
...     break
...     except ValueError:
...         print ("Debes introducir un número")
```

1. Se ejecuta el bloque de instrucciones de try.
2. Si no se produce la excepción, el bloque de except no se ejecuta y continúa la ejecución secuencia.
3. Si se produce una excepción, el resto del bloque try no se ejecuta, si la excepción que se produce corresponde con la indicada en except se salta a ejecutar el bloque de instrucciones except.
4. Si la excepción producida no se corresponde con las indicadas en except se pasa a otra instrucción try, si finalmente no hay un manejador nos dará un error y el programa terminará.

Un bloque except puede manejar varios tipos de excepciones, si quiero controlar varios tipos de excepciones puedo poner varios bloques except. Teniendo en cuenta que en el último, si quiero no indico el tipo de excepción:

```
>>> try:
...     print (10/int(cad))
... except ValueError:
...     print("No se puede convertir a entero")
... except ZeroDivisionError:
...     print("No se puede dividir por cero")
... except:
...     print("Otro error")
```

Se puede utilizar también la clausula else:

```
>>> try:
...     print (10/int(cad))
... except ValueError:
...     print("No se puede convertir a entero")
... except ZeroDivisionError:
...     print("No se puede dividir por cero")
... else:
...     print("Otro error")
```

Por último indicar que podemos indicar una clausula finally para indicar un bloque de instrucciones que siempre se debe ejecutar, independientemente de la excepción se haya producido o no.

```
>>> try:
...     result = x / y
... except ZeroDivisionError:
...     print("División por cero!")
... else:
...     print("El resultado es", result)
... finally:
...     print("Terminamos el programa")
```